Introduction to the I2C subsystem



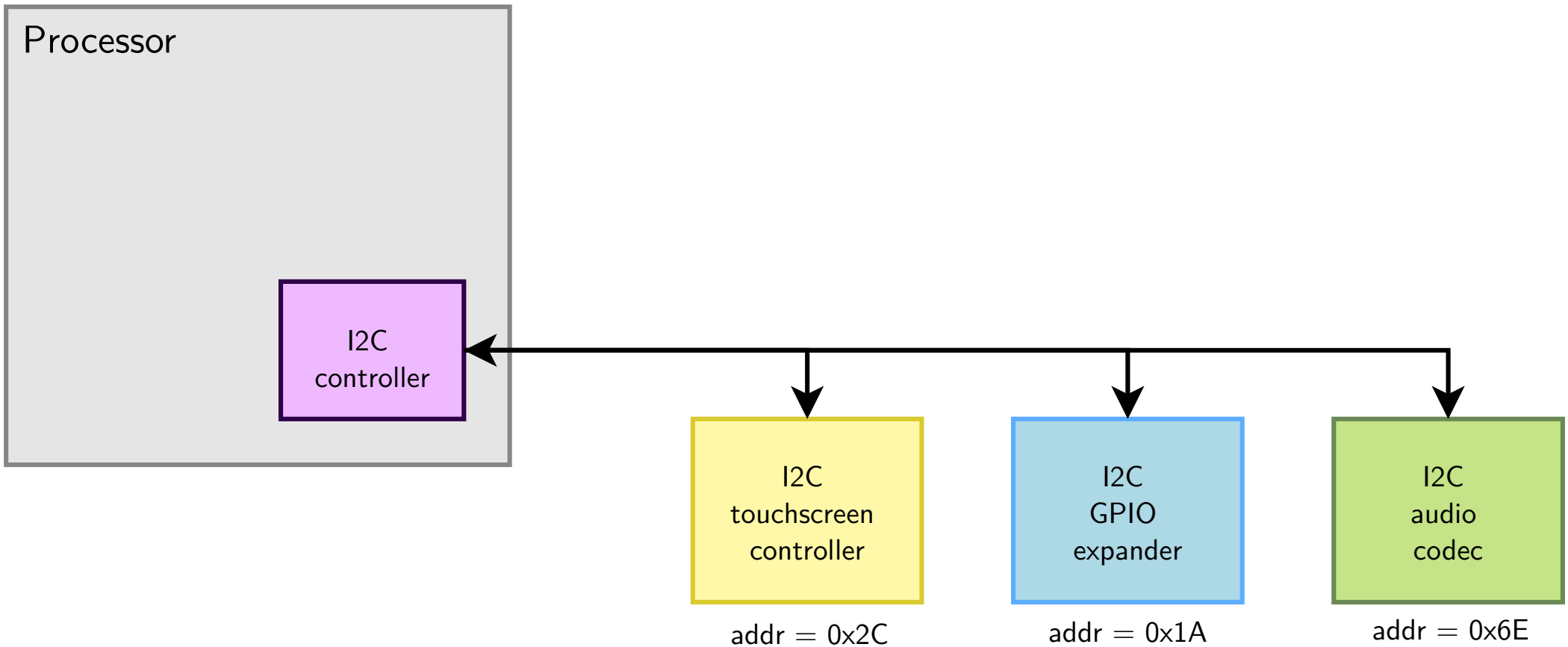
© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



What is I2C?

- A very commonly used low-speed bus to connect on-board and external devices to the processor.
- Uses only two wires: SDA for the data, SCL for the clock.
- It is a master/slave bus: only the master can initiate transactions, and slaves can only reply to transactions initiated by masters.
- In a Linux system, the I2C controller embedded in the processor is typically the master, controlling the bus.
- Each slave device is identified by an I2C address (you can't have 2 devices with the same address on the same bus). Each transaction initiated by the master contains this address, which allows the relevant slave to recognize that it should reply to this particular transaction.

An I2C bus example





The I2C bus driver

- Like all bus subsystems, the I2C bus driver is responsible for:
  - Providing an API to implement I2C controller drivers
  - Providing an API to implement I2C device drivers, in kernel space
  - Providing an API to implement I2C device drivers, in user space
- The core of the I2C bus driver is located in drivers/i2c/.
- The I2C controller drivers are located in drivers/i2c/busses/.
- The I2C device drivers are located throughout drivers/, depending on the framework used to expose the devices (e.g. drivers/input/ for input devices).



Registering an I2C device driver

- Like all bus subsystems, the I2C subsystem defines a struct i2c_driver that inherits from struct device_driver, and which must be instantiated and registered by each I2C device driver.
  - As usual, this structure points to the `->probe()` and `->remove()` functions.
  - It also contains a legacy `id_table`, used for non-DT based probing of I2C devices.
- The i2c_add_driver() and i2c_del_driver() functions are used to register/unregister the driver.
- If the driver doesn't do anything else in its `init()`/`exit()` functions, it is advised to use the module_i2c_driver() macro instead.



Registering an I2C device driver: example

```
static const struct i2c_device_id adxl345_i2c_id[] = {
        { "adxl345", ADXL345 },
        { "adxl375", ADXL375 },
        { }
};

MODULE_DEVICE_TABLE(i2c, adxl345_i2c_id);

static const struct of_device_id adxl345_of_match[] = {
        { .compatible = "adi,adxl345" },
        { .compatible = "adi,adxl375" },
        { },
};

MODULE_DEVICE_TABLE(of, adxl345_of_match);

static struct i2c_driver adxl345_i2c_driver = {
        .driver = {
                .name   = "adxl345_i2c",
                .of_match_table = adxl345_of_match,
        },
        .probe          = adxl345_i2c_probe,
        .remove         = adxl345_i2c_remove,
        .id_table       = adxl345_i2c_id,
};

module_i2c_driver(adxl345_i2c_driver);
```

From drivers/iio/accel/adxl345_i2c.c



Registering an I2C device: non-DT

- On non-DT platforms, the struct i2c_board_info structure allows to describe how an I2C device is connected to a board.
- Such structures are normally defined with the I2C_BOARD_INFO() helper macro.
  - Takes as argument the device name and the slave address of the device on the bus.
- An array of such structures is registered on a per-bus basis using i2c_register_board_info(), when the platform is initialized.



Registering an I2C device, non-DT example

arch/arm/mach-iop32x/em7210.c

```
static struct i2c_board_info __initdata em7210_i2c_devices[] = {
        { I2C_BOARD_INFO("rs5c372a", 0x32) },
};

static void __init em7210_init_machine(void)
{
        register_iop32x_gpio();
        platform_device_register(&em7210_serial_device);
        platform_device_register(&iop3xx_i2c0_device);
        platform_device_register(&iop3xx_i2c1_device);
        platform_device_register(&em7210_flash_device);
        platform_device_register(&iop3xx_dma_0_channel);
        platform_device_register(&iop3xx_dma_1_channel);

        i2c_register_board_info(0, em7210_i2c_devices,
                ARRAY_SIZE(em7210_i2c_devices));
}
```



Registering an I2C device, in the DT

- In the Device Tree, the I2C controller device is typically defined in the `.dtsi` file that describes the processor.
  - Normally defined with `status = "disabled"`.
- At the board/platform level:
  - the I2C controller device is enabled (`status = "okay"`)
  - the I2C bus frequency is defined, using the `clock-frequency` property.
  - the I2C devices on the bus are described as children of the I2C controller node, where the `reg` property gives the I2C slave address on the bus.
- See the binding for the corresponding driver for a specification of the expected DT properties. Example: Documentation/devicetree/bindings/i2c/ti,omap4-i2c.yaml



Registering an I2C device, DT example (1/2)

Definition of an I2C controller:

```
i2c0: i2c@01c2ac00 {
        compatible = "allwinner,sun7i-a20-i2c",
                     "allwinner,sun4i-a10-i2c";
        reg = <0x01c2ac00 0x400>;
        interrupts = <GIC_SPI 7 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&apb1_gates 0>;
        status = "disabled";
        #address-cells = <1>;
        #size-cells = <0>;
};
```

From arch/arm/boot/dts/allwinner/sun7i-a20.dtsi

`#address-cells`: number of 32-bit values needed to encode the address fields

`#size-cells`: number of 32-bit values needed to encode the size fields

See details in https://elinux.org/Device_Tree_Usage



Registering an I2C device, DT example (2/2)

Definition of the I2C device

```
&i2c0 {
        pinctrl-names = "default";
        pinctrl-0 = <&i2c0_pins_a>;
        status = "okay";

        axp209: pmic@34 {
                compatible = "x-powers,axp209";
                reg = <0x34>;
                interrupt-parent = <&nmi_intc>;
                interrupts = <0 IRQ_TYPE_LEVEL_LOW>;

                interrupt-controller;
                #interrupt-cells = <1>;
        };
};
```

From arch/arm/boot/dts/allwinner/sun7i-a20-olinuxino-micro.dts



`probe()` and `remove()`

- The `->probe()` function is responsible for initializing the device and registering it in the appropriate kernel framework. It receives as argument:
  - An struct i2c_client pointer, which represents the I2C device itself. This structure inherits from struct device.
  - On older kernels ($<$ v6.4), `->probe()` was taking a second (unused) argument, the removal of this other argument implied the use of another probe function for some kernel releases, called `->probe_new()`.
- The `->remove()` function is responsible for unregistering the device from the kernel framework and shut it down. It receives as argument:



Probe example

```
static int da311_probe(struct i2c_client *client)
{
        struct iio_dev *indio_dev;          // framework structure
        da311_data *data;                   // per device structure
        ...
        // Allocate framework structure with per device struct inside
        indio_dev = devm_iio_device_alloc(&client->dev, sizeof(*data));
        data = iio_priv(indio_dev);
        data->client = client;
        i2c_set_clientdata(client, indio_dev);
        // Prepare device and initialize indio_dev
        ...
        // Register device to framework
        ret = iio_device_register(indio_dev);
        ...
        return ret;
}
```

From drivers/iio/accel/da311.c



Remove example

```
static int da311_remove(struct i2c_client *client)
{
        struct iio_dev *indio_dev = i2c_get_clientdata(client);
        // Unregister device from framework
        iio_device_unregister(indio_dev);
        return da311_enable(client, false);
}
```

From drivers/iio/accel/da311.c



Communicating with the I2C device: raw API

The most **basic API** to communicate with the I2C device provides functions to either send or receive data:

- Send a `buf` to the I2C device with:

```
int i2c_master_send(const struct i2c_client *client, const char *buf, int count);
```

- Receive a `count` bytes from the I2C device and save them in `buf` with:

```
int i2c_master_recv(const struct i2c_client *client, char *buf, int count);
```

Both functions return a negative error number in case of failure, otherwise the number of transmitted bytes.



Communicating with the I2C device: message transfer

The message transfer API allows to describe **transfers** that consists of several **messages**, with each message being a transaction in one direction:

```
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg *msgs, int num);
```

- The struct i2c_adapter pointer can be found by using `client->adapter`
- The struct i2c_msg structure defines the length, location, and direction of the message.



I2C: message transfer example

```
static int st1232_ts_read_data(struct st1232_ts_data *ts)
{
        ...
        struct i2c_client *client = ts->client;
        struct i2c_msg msg[2];
        int error;
        ...
        u8 start_reg = ts->chip_info->start_reg;
        u8 *buf = ts->read_buf;

        /* read touchscreen data */
        msg[0].addr = client->addr;
        msg[0].flags = 0;
        msg[0].len = 1;
        msg[0].buf = &start_reg;

        msg[1].addr = client->addr;
        msg[1].flags = I2C_M_RD;
        msg[1].len = ts->read_buf_len;
        msg[1].buf = buf;

        error = i2c_transfer(client->adapter, msg, 2);
        ...
}
```

From drivers/input/touchscreen/st1232.c



I2C functionality

- Not all I2C controllers support all functionalities.
- The I2C controller drivers therefore tell the I2C core which functionalities they support.
- An I2C device driver must check that the functionalities they need are provided by the I2C controller in use on the system.
- The i2c_check_functionality() function allows to make such a check.
- Examples of functionalities: I2C_FUNC_I2C to be able to use the raw I2C functions, I2C_FUNC_SMBUS_BYTE_DATA to be able to use SMBus commands to write a command and read/write one byte of data.
- See include/uapi/linux/i2c.h for the full list of existing functionalities.



References

- https://en.wikipedia.org/wiki/I2C, general presentation of the I2C protocol
- i2c/, details about Linux support for I2C
  - i2c/writing-clients
    How to write I2C kernel device drivers
  - i2c/dev-interface
    How to write I2C user-space device drivers
  - i2c/instantiating-devices
    How to instantiate devices
  - i2c/smbus-protocol
    Details on the SMBus functions
  - i2c/functionality
    How the functionality mechanism works
- See also Luca Ceresoli's introduction to I2C (slides, video).

Practical lab - Communicate with the Nunchuk



- Explore the content of `/dev` and `/sys` and the devices available on the embedded hardware platform.
- Implement a driver that registers as an I2C driver.
- Communicate with the Nunchuk and extract data from it.